Memoria de uso de IA

SmartCampus · Proyecto Big Data I · Taller evaluable 01 · Nivel AMARILLO

Equipo	Luis Wesinger · Yiao Tang
Repositorio	https://github.com/luiswesinger/practica01
Herramienta utilizada	DeepSeek
Alcance de este registro	Consultas sobre **Docker** y sobre **GitHub**, incluida la resolución de errores de ejecución y de construcción
Fecha	14 de octubre de 2026

---

1 · Declaración y alcance

Se declara el uso de un asistente de inteligencia artificial generativa (DeepSeek) durante el desarrollo del taller. Su uso se limitó a **consultas puntuales sobre Docker y sobre GitHub**: dudas de configuración de los servicios y, sobre todo, **diagnóstico de errores** que aparecieron al construir las imágenes y al levantar el entorno.

En particular, se consultó al asistente sobre:

- La configuración de red y de puertos entre los contenedores.
- Errores de construcción de las imágenes y su causa.
- Errores de inicialización de la base de datos.
- El acceso a la cuenta y al repositorio de GitHub, y la sincronización entre los equipos del grupo.

El diseño y la implementación de la infraestructura son obra del equipo. Los ficheros `docker-compose.yml`, los tres `Dockerfile`, el esquema `db/init.sql` y la configuración de la red y del volumen se redactaron y se probaron por el equipo. La IA se utilizó como apoyo para resolver dudas y errores concretos, no para producir el código de la entrega.

> **Alcance no cubierto por esta memoria.** El desarrollo del notebook de análisis (`notebooks/analyse_data.ipynb`) corresponde al Reto B y se documenta en la memoria de uso de IA

de su responsable, que se adjunta por separado. El presente registro recoge únicamente las consultas relativas a Docker y a GitHub.

2 · Herramientas y finalidades

Herramienta	Integrante	Finalidad	Ámbito
DeepSeek	Yiao Tang	Verificación del acceso a la cuenta y al repositorio de GitHub	GitHub
DeepSeek	Yiao Tang	Dudas sobre la red interna, los puertos y el descubrimiento de servicios	Docker
DeepSeek	Yiao Tang	Diagnóstico de errores de construcción de las imágenes	Docker
DeepSeek	Yiao Tang	Diagnóstico de conflictos de puertos en el equipo anfitrión	Docker
DeepSeek	Yiao Tang	Diagnóstico de errores de inicialización de PostgreSQL	Docker
DeepSeek	Yiao Tang	Resolución de problemas de sincronización con el repositorio	GitHub

3 · Índice cronológico de los prompts

ID	Ámbito	Objetivo	Resultado
P-01	GitHub	Comprobar el acceso a la cuenta y al repositorio del equipo	Funcionó
P-02	Docker	Resolver dudas sobre red interna, puertos y nombres de servicio	Funcionó
P-03	Docker	Diagnosticar un error de permisos al instalar dependencias en el build	Funcionó
P-04	Docker	Diagnosticar un conflicto de puertos en el equipo anfitrión	Funcionó
P-05	Docker	Diagnosticar que la tabla no existía al arrancar	Funcionó
P-06	GitHub	Resolver un rechazo al subir cambios tras reescribir el historial	Funcionó

4 · Fichas de interacción

P-01 · Acceso a la cuenta y al repositorio (GitHub)

Objetivo	Comprobar si la cuenta del equipo tenía acceso al repositorio creado por el compañero, antes de empezar a trabajar.
Consulta	Verificación del estado de la cuenta, del repositorio y de los permisos disponibles.
Resultado obtenido	Se confirmó que la cuenta existe, que el repositorio es accesible y que se dispone de permiso de escritura. Se detectó además que el repositorio contenía únicamente el material original del profesorado, sin ninguna implementación.
Problemas o limitaciones	Ninguno. La comprobación evitó empezar a trabajar sobre una copia local desactualizada.
Comprobación independiente	Verificación directa contra la API de GitHub y comprobación de los permisos efectivos sobre el repositorio.
Se integró	Sí: a partir de esta comprobación se clonó el repositorio y se configuró la identidad de Git **únicamente para ese repositorio**, ya que la configuración global del equipo apuntaba a otra cuenta y los commits no se habrían atribuido correctamente.

P-02 · Red interna, puertos y nombres de servicio (Docker)

Objetivo	Decidir si el servicio de base de datos debía publicar su puerto al equipo anfitrión y aclarar cómo se localizan los servicios entre sí.
Resultado obtenido	Se confirmó que los contenedores de una misma red de Compose se resuelven **por el nombre del servicio**, y que `localhost` apunta al propio contenedor. Se concluyó que el puerto de PostgreSQL no necesita publicarse, porque solo lo consumen los otros dos servicios a través de la red interna.
Decisión del equipo	Se decidió **no publicar** el puerto de la base de datos. El enunciado pide exponer «los puertos necesarios para acceder a Jupyter y a la API», y PostgreSQL no entra en ese conjunto. No publicarlo evita además conflictos con otros PostgreSQL ya instalados en la máquina.
Comprobación independiente	Se verificó con `docker compose ps` que el servicio `db` muestra `5432/tcp` sin mapeo al host, y que desde el contenedor de Jupyter la conexión a `db:5432` funciona correctamente.

P-03 · Error de permisos al instalar dependencias (Docker)

Error observado	``` Step 3/6 : RUN pip install --no-cache-dir -r /tmp/requirements.txt ERROR: Could not open requirements file: [Errno 13] Permission denied: '/tmp/requirements.txt' ```
Causa identificada	Dos hechos que se combinan: la instrucción `COPY` **conserva los permisos del fichero original** del equipo anfitrión, y la imagen base de Jupyter **no ejecuta los comandos como** `root` sino como el usuario `jovyan`. El fichero llegaba con modo `600` y propietario `root`, así que `pip` no podía leerlo.
Intento que no funcionó	Añadir `RUN chmod 0644` tras el `COPY` también fallaba, con `Operation not permitted`, porque el propio `RUN` se ejecuta como `jovyan`, que no es el propietario del fichero copiado.
Corrección aplicada por el equipo	Ceder la propiedad en la propia instrucción de copia: ``` COPY --chown=jovyan:users requirements.txt /tmp/requirements.txt ``` De este modo el fichero es legible por quien ejecuta `pip`, con independencia del modo con que esté en cada máquina.
Comprobación independiente	Se reconstruyó la imagen y se ejecutó el contenedor para verificar que las dependencias están presentes dentro de la imagen y que el entorno arranca sin instalar nada manualmente.
Se integró	Sí, en `jupyter/Dockerfile` y `api/Dockerfile`.

P-04 · Conflicto de puertos en el equipo anfitrión (Docker)

Error observado	El servicio de la API no arrancaba por puerto ya en uso. La comprobación del puerto mostraba que lo ocupaba un proceso del sistema, no otro contenedor.
	``
	$ lsof -nP -iTCP:5000 -sTCP:LISTEN
	ControlCe 1134 yiao ... TCP *:5000 (LISTEN)
	``

Causa identificada	En macOS el puerto 5000 lo ocupa el sistema: el «Receptor de AirPlay» (Centro de Control). No era un conflicto con otro servicio del proyecto.

Corrección aplicada por el equipo	Se optó por **cambiar únicamente el puerto publicado** —el lado izquierdo del mapeo— sin tocar el puerto interno del contenedor, que es el que usa Flask: `API_PORT=5001  # en el fichero .env local` Se documentó en `.env.example` para que cualquier miembro del equipo pueda resolverlo.
Comprobación independiente	Se verificó que el servicio arranca, que `GET /health` responde correctamente y que el puerto interno del contenedor sigue siendo el esperado.
Nota	El fichero `.env` está excluido del repositorio, de modo que esta configuración es local y no afecta a los demás equipos.

P-05 · La tabla no existe al arrancar (Docker)

Error observado	`ERROR:  relation "lecturas" does not exist`
Causa identificada	El esquema se define en `db/init.sql`, que la imagen de PostgreSQL ejecuta **únicamente cuando inicializa un volumen de datos vacío**. Si el volumen ya existía de una ejecución anterior, el script no se vuelve a ejecutar.

Corrección aplicada por el equipo	Se eliminó el volumen para forzar una inicialización limpia y se volvió a levantar el entorno:
	`docker compose down -v`
	`docker compose up --build`

Comprobación independiente	Se verificó la creación de la tabla y de sus dos índices consultando el esquema directamente en el contenedor de la base de datos.
Se documentó	Sí, en el README y en la documentación técnica del proyecto, porque es un comportamiento que sorprende y conviene conocer.

P-06 · Rechazo al subir cambios tras reescribir el historial (GitHub)

Error observado	La subida de cambios fue rechazada porque el historial local y el remoto habían divergido.
Causa identificada	Los commits de una de las ramas de trabajo se habían reorganizado, de modo que el historial local y el del repositorio remoto ya no compartían la misma secuencia.
Corrección aplicada por el equipo	Se sincronizó el repositorio local con el remoto antes de continuar y, para el compañero, se documentó el procedimiento aplicable a cada situación (sin copia local, con copia sin cambios y con cambios propios sin confirmar).
Comprobación independiente	Se verificó que el historial remoto quedaba coherente y que el contenido de los ficheros era idéntico al del equipo local antes de la operación.
Prevención	Se acordó evitar reorganizar el historial una vez compartido, y avisar al equipo cuando fuera imprescindible.

5 · Balance final

5.1 · En qué resultó útil la IA

La utilidad se concentró en el **diagnóstico de errores**. Dos de ellos tenían causas poco evidentes que habrían costado bastante localizar por prueba y error: el fallo de permisos en la construcción de la imagen de Jupyter —originado en que `COPY` conserva los permisos del fichero original mientras que la imagen base no ejecuta como `root` — y el hecho de que `init.sql` solo se ejecute sobre un volumen vacío. También resultó práctico para **confirmar cómo se resuelven los servicios** dentro de la red de Compose.

5.2 · Qué respuestas no sirvieron o generaron problemas

- **Una solución que no resolvía el problema.** Para el error de permisos se propuso primero añadir un `chmod` después del `COPY`. La propuesta era razonable pero fallaba igualmente, con `Operation not permitted`, porque el propio comando se ejecuta con el usuario sin privilegios. Hubo que reformular la consulta indicando ese segundo error para llegar a la causa real.
- **Un diagnóstico incompleto del conflicto de puertos.** La primera respuesta asumió que el puerto lo ocupaba otro contenedor del proyecto. La comprobación con `lsof` mostró que lo ocupaba un proceso del propio sistema operativo.

5.3 · Decisiones, comprobaciones y correcciones del equipo

- **Diseñar y redactar** el `docker-compose.yml`, los tres `Dockerfile` y el esquema inicial: la IA no generó estos ficheros.
- **Decidir no publicar** el puerto de PostgreSQL al equipo anfitrión.
- **Comprobar** que la tabla y los dos índices definidos en `init.sql` siguen intactos después de cargar los datos.
- **Verificar** que los servicios arrancan, que los tres contenedores quedan activos y que la comunicación entre ellos funciona por nombre de servicio.
- **Comprobar la persistencia** de los datos tras reiniciar los contenedores.
- **Documentar** las incidencias y sus soluciones para que cualquier miembro del equipo pueda resolverlas.

5.4 · Qué haríamos de forma diferente

Se aportaría desde el primer momento el **texto completo del error** y el contexto de la configuración, en lugar de una descripción del síntoma: las dos respuestas que no sirvieron se debieron a que la consulta inicial no incluía información suficiente para acotar la causa. También conviene **comprobar cada propuesta ejecutándola** antes de darla por válida, ya que la primera solución al error de permisos parecía correcta sobre el papel y no funcionaba.

6 · Verificación de la trazabilidad

☑	Están todos los prompts de consulta sobre Docker y GitHub, incluidos los reintentos y las respuestas que no sirvieron.
☑	Cada prompt tiene su captura legible con el texto y la respuesta recibida.
☑	Se indica qué funcionó, qué no y cómo se comprobó.
☑	Se identifican los cambios propios, las pruebas y los resultados verificables.
☑	No aparecen contraseñas, tokens ni datos personales en texto plano.
☑	El equipo puede explicar y defender la configuración y los resultados de la entrega.

7 · Procedencia del registro

La conversación se obtuvo con la **función de exportación a PDF** integrada en la propia interfaz del asistente, no mediante capturas manuales ni reconstrucciones posteriores. El fichero exportado conserva la dirección de la conversación y la marca de tiempo en su pie de página, lo que permite verificar su origen. Dicho fichero se adjunta junto a esta memoria.
